



**Object Oriented
Programming**



Programação Orientado a Objetos

Encapsulamento em Java: A Interface do seu Objeto

Prof. Luís Guilherme de S. Lopes

Os Três Pilares da POO:

- **Encapsulamento:** Esconder detalhes de implementação, expondo apenas o necessário.
- **Herança:** Criar novas classes a partir de classes existentes, reaproveitando código.
- **Polimorfismo:** Permitir que objetos de diferentes classes respondam a mesma mensagem de forma diferente.

Encapsulamento

O que é?

Ocultar partes internas do objeto, expondo apenas uma interface pública.

Analogia:

O controle remoto da TV, você interage com os botões, sem saber como ele funciona por dentro.

Interface

O que é?

- A "fachada" do objeto, o que ele mostra para o mundo exterior.
- Define os métodos que podem ser usados para interagir com o objeto.
- Garante que o objeto seja usado de forma correta e segura.

Vantagens do Encapsulamento

Tornar mudanças invisíveis:

Modificar a implementação interna sem afetar o código que usa o objeto.

Analogia:

Trocar a pilha do controle remoto sem que ele pare de funcionar.

Vantagens do Encapsulamento

Facilitar a reutilização do código:

Criar componentes reutilizáveis em diferentes projetos.

Analogia:

Usar a mesma bateria em diferentes dispositivos.

Vantagens do Encapsulamento

Reduzir efeitos colaterais:

- Evitar que alterações em uma parte do código afetem outras partes inesperadamente.
- Proteger os dados do objeto de acessos e modificações indevidas.

Exemplo de Encapsulamento (Controle Remoto)

Interface Controlador

Define os métodos que um controle remoto deve ter:
`ligar()`, `desligar()`, `abrirMenu()`, etc.

Métodos Construtores (Construct)

Classe ControleRemoto

- Implementa a interface **Controlador**, fornecendo a implementação concreta dos métodos.
- Usa atributos privados (**volume**, **ligado**, **passandoPrograma**) para armazenar o estado interno.
- Usa getters e setters para acessar e modificar os atributos de forma controlada.

Código da Interface Controlador

```
public interface Controlador {  
    public abstract void ligar();  
    public abstract void desligar();  
    public abstract void abrirMenu();  
    public abstract void fecharMenu();  
    public abstract void maisVolume();  
    public abstract void menosVolume();  
    public abstract void ligarMudo();  
    public abstract void desligarMudo();  
    public abstract void play();  
    public abstract void pause();  
}
```

Código da Classe ControleRemoto (parcial)

```
public class ControleRemoto implements Controlador {  
    // Atributos privados  
    private int volume;  
    private boolean ligado;  
    private boolean passandoprograma;  
  
    // Construtor  
    public ControleRemoto() {  
        this.volume = 50;  
        this.ligado = true;  
        this.passandoprograma = false;  
    }  
  
    // Getters e setters  
    // ...  
  
    // Métodos abstratos da interface  
    @Override  
    public void ligar() {  
        this.setLigado(true);  
    }  
  
    // ... outros métodos  
}
```

Conclusão

Recapitulação:

- Encapsulamento é a base para criar código seguro, reutilizável e fácil de manter.
- Interfaces definem o contrato que as classes devem seguir.
- Getters e setters permitem controlar o acesso aos atributos privados.



VAMOS
PRATICAR!